

Reviewer Pack — Free Cumulant Sum Lower Bounded by Communication Complexity ...

Entry f61b226b5cec · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-13 08:49:42 UTC

Free Cumulant Sum Lower Bounded by Communication Complexity of DISJOINTNESS

Entry ID: f61b226b5cec **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-13 08:49:42 UTC

1. Conjecture

Field A (mathematical branch): Free Probability Theory **Field B** (complexity object): Randomized Communication Complexity of DISJOINTNESS

Statement:

For a $|X| \times |Y|$ matrix M defining DISJOINTNESS, let $\tau(M) = \sum_{k=1}^{\infty} |\gamma_k(M)|$ where γ_k are free cumulants of the non-commutative distribution induced by M . Then $\tau(M) \geq c \cdot n$ for some absolute constant $c > 0$, with equality achieved by the standard DISJOINTNESS matrix.

Rationale (proposer's reasoning):

Free cumulants capture non-commutative dependencies in distributions, which may mirror the entanglement structure required for communication in DISJOINTNESS. The sum's growth rate could expose a quantitative lower bound via analytic continuation of the R-transform.

Taxonomy category: COMM_DISJ (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 273d2b255da54282

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def gaussian_elimination(A, b):
        n = len(b)
        for i in range(n):
            max_row = i + max(range(i, n), key=lambda j: abs(A[j][i]))
            A[i], A[max_row] = A[max_row], A[i]
            b[i], b[max_row] = b[max_row], b[i]
            factor = 1 / A[i][i]
            for j in range(n):
                if i != j:
                    factor_i_j = A[j][i] * factor
                    A[j] = [A[j][k] - factor_i_j * A[i][k] for k in range(n)]
                    b[j] -= factor_i_j * b[i]
            return [b[i] / A[i][i] for i in range(n)]

    def inverse_matrix(M):
        n = len(M)
        I = [[1 if i == j else 0 for j in range(n)] for i in range(n)]
        return gaussian_elimination(M, I)

    def r_transform_inv(M):
        n = len(M)
        M_inv = inverse_matrix(M)
        R = [[0] * n for _ in range(n)]
        for i in range(n):
            for j in range(n):
                if i != j:
                    factor = 1 / (M[i][i] - M[j][j])
                    R[i][j] = factor * (M_inv[i][j] + sum(M_inv[k][k] * M_inv[i][k] * M_inv[j][k]
                    for k in range(n)))
            R[i][i] = 1 / M[i][i]
        return R

    def free_cumulant_sum(R):
        n = len(R)
```

```

tau = 0
for k in range(1, n + 1):
    term = sum(math.comb(k - 1, j) * math.prod([R[i][i] ** (k - j - 1) for i in range(n)])
    tau += term / k
return tau

def generate_disjointness_matrix(n):
    M = [[0] * n for _ in range(n)]
    for i in range(n):
        M[i][i] = 1
    return M

n = random.randint(5, 40)
M = generate_disjointness_matrix(n)
R = r_transform_inv(M)
tau = free_cumulant_sum(R)

return {
    "metric_name": "tau",
    "metric_value": tau,
    "instances_tested": 1,
    "conjecture_holds": tau >= 0.3 * n,
    "counterexample": "" if tau >= 0.3 * n else "disjointness_matrix"
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 1 for i in range(5, 8)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_tau = sum(r["metric_value"] for r in results) / len(results)
    std_tau = math.sqrt(sum((r["metric_value"] - mean_tau) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_tau} std={std_tau} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_tau} std={std_tau} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(seed for seed, r in zip(seeds, results) if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='disjointness_matrix' first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_63538252.py", line 84, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_63538252.py", line 67, in run_trial
    R = r_transform_inv(M)
          ^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_63538252.py", line 42, in r_transform_inv
    M_inv = inverse_matrix(M)
              ^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_63538252.py", line 38, in inverse_matrix
    return gaussian_elimination(M, I)
           ^^^^^^^^^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_63538252.py", line 32, in gaussian_elimination
    b[j] -= factor_i_j * b[i]
              ^~~~~~
TypeError: can't multiply sequence by non-int of type 'float'

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed during matrix inversion with type error, preventing result collection | next:
Fix the TypeError in Gaussian elimination by ensuring compatible data types for matrix operations

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	18372
2	preregistration	ollama_remote	qwen3:8b	0	0	25807
3	novelty	ollama_remote	qwen3:8b	0	0	21987
4	novelty	ollama_remote	qwen3:8b	0	0	13056
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14541
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10299
7	judge	ollama_remote	qwen3:8b	0	0	16663

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 120725 ms total latency. Provider mix: {'ollama_remote': 7}

(full prompt+response transcripts available in `research/audit/f61b226b5cec.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/f61b226b5cec.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/f61b226b5cec.tar.gz` (if generated)